
Hermes Automation

Guia da API REST v1

Integração machine-to-machine

30 de abril de 2026

1. Visão geral

A API REST do Hermes expõe o pipeline de automação TISS para sistemas externos. Um caller pode disparar o fluxo completo (parse do XML, validação, login no portal Orizon Fature e submissão do lote) sem passar pela UI logada.

Endpoints públicos vivem sob `/api/v1/*`. Toda chamada exige uma API key Bearer. Recursos (jobs, credenciais, eventos) são estritamente escopados ao dono da chave — não há tenant compartilhado.

Base URL: `https://<seu-host>/api/v1/...` (em desenvolvimento, `http://localhost:3000/api/v1/...`).

Audiência: backend developer integrando o Hermes a um ERP, RIS ou outra ferramenta de gestão clínica que já produz XMLs TISS.

2. Autenticação

Toda requisição precisa do header `Authorization: Bearer hapi_...`. Sem ele, a API responde 401 UNAUTHORIZED.

Como criar uma chave

1. Faça login na UI com um usuário aprovado.
2. Vá em `/app/settings/api-keys`.
3. Clique em *Gerar chave*, informe um rótulo (ex.: "Sistema XYZ — produção") e confirme.
4. Copie o segredo **imediatamente**. Ele só aparece uma vez — o servidor armazena apenas o `sha256(secret)`; a versão em texto puro é descartada.

Formato do segredo

`hapi_` + 32 caracteres base32 minúsculos (ex.: `hapi_esvxo23eblyqcpgiol6voc7hfpl4ptre`). Os primeiros 13 caracteres formam o *prefix* exibido na UI para identificar a chave.

Revogação

A revogação é imediata e definitiva — basta clicar no ícone de lixeira na linha da chave. Aplicações que estiverem usando aquela chave receberão 401 na próxima chamada. Não é possível desfazer.

3. Modelo conceitual

3.1 Job lifecycle

Um **job** representa o processamento de 1 ou mais arquivos TISS de um único lote operacional. O campo status evolui assim:

Status	Quem seta	Próximo passo típico
uploaded	POST /jobs (criação)	Workflow começa o ingestTiss → awaiting_validation
awaiting_validation	tool requestHumanValidation	Aprovação humana ou auto-aprovação
approved	Resume validation (humano ou Loginto)	envio TISS começam → running
running	tool fillOrizonCredentials	Termina em login_succeeded ou failed
login_succeeded	Login + submit OK	Terminal (sucesso)
failed	Qualquer erro fatal ou cancel	Terminal (falha)

3.2 Flow types

short (default): faz upload do(s) .zip na tela *Enviar XML TISS*, marca os lotes válidos e confirma. É o fluxo típico para produção em massa.

complete: abre a página *Digital Guia* e preenche cada guia campo a campo. Útil quando o portal recusa o upload em lote ou exige edição manual.

3.3 Platform credentials

São as credenciais que o agente usa para logar no portal alvo. A senha é cifrada com AES-256-GCM (CREDENTIAL_ENCRYPTION_KEY) e nunca volta para o caller — você só vê usernameMasked. Trocar a chave de criptografia invalida todas as senhas armazenadas; quem fizer isso precisa re-salvar todas as credenciais.

3.4 Auto-approve vs validação humana

Se você passa platformCredentialId no POST do job, o workflow detecta isso, pula a tool de aprovação humana e segue direto para o login. O evento validation_approved é emitido com payload .autoApproved=true para deixar isso claro na auditoria. Se você omite o campo, o workflow pausa e fica aguardando que um humano aprove pela UI antes de prosseguir — comportamento idêntico ao da UI atual.

4. Endpoints

POST /api/v1/jobs

Cria um job e enfileira o workflow.

Request

Content-Type: multipart/form-data

Campos:

file	arquivo .zip ou .xml (repetir N vezes; 1-50)	
flowType	short complete	[default: short]
platformId	orizon_fature	[default: orizon_fature]
platformCredentialId	uuid (opcional). Se enviado, pula validacao humana.	

Response

```
{
  "ok": true,
  "jobId": "uuid",
  "runId": "string",
  "fileCount": 2,
  "autoApproved": true
}
```

Exemplo

```
curl -X POST http://localhost:3000/api/v1/jobs \
  -H "Authorization: Bearer $KEY" \
  -F "flowType=short" \
  -F "platformCredentialId=<credId>" \
  -F "file=@./lote_1124.zip" \
  -F "file=@./lote_1122.zip"
```

GET /api/v1/jobs

Lista os 50 jobs mais recentes do dono da API key.

Request

(sem corpo)

Response

```
{
  "ok": true,
  "jobs": [
    { "id": "...", "status": "login_succeeded", "fileCount": 3, "tiss": {...}, ... }
  ]
}
```

Exemplo

```
curl -H "Authorization: Bearer $KEY" \
  http://localhost:3000/api/v1/jobs
```

GET /api/v1/jobs/{jobId}

Detalhe completo: status, files[], TISS summary, eventos e estado do workflow.

Request

(sem corpo)

Response

```
{
```

```

    "ok": true,
    "job": { ... colunas da tabela jobs ... },
    "files": [ { "fileName": "...", "size": "...", "checksum": "..." } ],
    "tiss": { "standardVersion": "4.02.00", "guideCount": "19", ... },
    "events": [ { "type": "...", "message": "...", "payload": {...} } ],
    "workflow": { "nodes": [...], "edges": [...], "currentNode": "..." }
}

```

Exemplo

```

curl -H "Authorization: Bearer $KEY" \
  http://localhost:3000/api/v1/jobs/$JOB_ID

```

POST /api/v1/jobs/{jobId}/cancel

Cancela o workflow se o job ainda estiver em execucao. Idempotente.

Request

(sem corpo)

Response

```

{ "ok": true }
ou
{ "ok": true, "alreadyTerminal": true, "status": "login_succeeded" }

```

Exemplo

```

curl -X POST -H "Authorization: Bearer $KEY" \
  http://localhost:3000/api/v1/jobs/$JOB_ID/cancel

```

GET /api/v1/jobs/{jobId}/events

Stream Server-Sent Events ligado ao workflow run.

Request

Query opcional: startIndex (replay a partir do indice).

Response

Content-Type: text/event-stream

```

data: {"type":"agent_tool_completed","message":"...","payload":{...}}
data: {"type":"submit_tiss_progress","message":"...","payload":{...}}
...

```

Retorna 409 RUN_NOT_READY se o workflow ainda nao iniciou.

Exemplo

```

curl -N -H "Authorization: Bearer $KEY" \
  http://localhost:3000/api/v1/jobs/$JOB_ID/events

```

GET /api/v1/platform-credentials

Lista as credenciais cadastradas pelo dono da API key.

Request

(sem corpo)

Response

```

{
  "ok": true,
  "credentials": [

```

```
{ "id": "...", "platformId": "orizon_fature", "label": "Clinica X",
  "usernameMasked": "us***om", "createdAt": "...", "updatedAt": "..." }
]
```

Exemplo

```
curl -H "Authorization: Bearer $KEY" \
  http://localhost:3000/api/v1/platform-credentials
```

POST /api/v1/platform-credentials

Cria uma credencial. A senha é cifrada com AES-256-GCM no servidor.

Request

Content-Type: application/json

```
{
  "platformId": "orizon_fature",
  "label": "Clinica X", (2-80 chars)
  "username": "186870", (1-160 chars)
  "password": "SEGREDO" (1-512 chars; nunca retornado)
}
```

Response

```
{
  "ok": true,
  "credential": {
    "id": "uuid",
    "platformId": "orizon_fature",
    "label": "Clinica X",
    "usernameMasked": "18***70"
  }
}
```

Exemplo

```
curl -X POST http://localhost:3000/api/v1/platform-credentials \
  -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" \
  -d '{"platformId":"orizon_fature","label":"Clinica X","username":"186870","password":"SEGREDO"}'
```

GET /api/v1/platform-credentials/{id}

Retorna metadados de uma credencial. Nunca expoe a senha.

Request

(sem corpo)

Response

```
{
  "ok": true,
  "credential": { "id": "...", "label": "...", "usernameMasked": "..." }
}
```

Exemplo

```
curl -H "Authorization: Bearer $KEY" \
  http://localhost:3000/api/v1/platform-credentials/$CRED_ID
```

PATCH /api/v1/platform-credentials/{id}

Atualiza label, username ou password (qualquer subconjunto).

Request

Content-Type: application/json

```
{
  "label":      "Clinica X (renomeada)",    (opcional)
  "username":   "186871",                  (opcional)
  "password":   "NOVO_SEGREDO"             (opcional; re-cifra)
}
```

Response

```
{
  "ok": true,
  "credential": { "id": "...", "label": "...", "usernameMasked": "..."}
}
```

Exemplo

```
curl -X PATCH http://localhost:3000/api/v1/platform-credentials/$CRED_ID \
  -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" \
  -d '{"password":"NOVO_SEGREDO"}'
```

DELETE /api/v1/platform-credentials/{id}

Remove a credencial. Falha com 409 se algum job ainda referencia.

Request

(sem corpo)

Response

```
{ "ok": true }
ou
{ "ok": false, "error": { "code": "CREDENTIAL_IN_USE", "message": "..."} }
```

Exemplo

```
curl -X DELETE -H "Authorization: Bearer $KEY" \
  http://localhost:3000/api/v1/platform-credentials/$CRED_ID
```

5. Eventos do timeline

Toda chamada de tool, transição de status e progresso de browser produz um evento gravado em `job_events` e replicado no SSE stream. A tabela abaixo lista os tipos mais comuns; o campo `payload` traz metadados específicos (status, `sessionId`, `autoApproved`, etc.).

type	Significado
uploaded	Arquivo TISS recebido pelo storage.
agent_started	Workflow durable começou a rodar o agente.
agent_tool_called	Agente invocou uma tool (<code>ingestTiss</code> , <code>fillOrizonCredentials</code> , ...).
agent_tool_completed	Tool terminou (sucesso ou falha capturada na payload).
agent_step_started	Novo step de raciocínio do agente.
human_validation_requested	Workflow pausou aguardando aprovação humana na UI.
validation_approved	Validação aprovada. <code>payload.autoApproved=true</code> se veio via API.
browser_session_started	Sessão Browserbase realmente criada (<code>sessionId</code> no payload).
browser_action_completed	Etapa de browser concluiu (<code>login/submit</code>). <code>status=success failed</code> .
submit_tiss_progress	Update intermediário do envio (<code>file_uploaded</code> , <code>batches_selected</code> , ...).
submit_tiss_completed	Lote TISS confirmado pelo portal.
submit_tiss_failed	Envio do lote não confirmou.
agent_completed	Workflow encerrou (sucesso ou falha).
failed	Job marcado como falho fora do agente (ex: cancel manual).

6. Erros

Erros sempre vêm no envelope { ok: false, error: { code, message } }. O code é estável e pode ser usado para roteamento programático.

code	HTTP	Significado
UNAUTHORIZED	401	Authorization ausente, malformado, revogado ou expirado.
PENDING_APPROVAL	403	Dona(o) da API key nao esta com status approved.
NOT_FOUND	404	Recurso inexistente ou nao pertence ao dono da API key.
INVALID_BODY	400	JSON invalido ou campo obrigatorio faltando.
FILE_REQUIRED	400	POST /jobs sem arquivo no multipart.
TOO_MANY_FILES	400	Mais de 50 arquivos por request.
CREDENTIAL_NOT_FOUND	400	platformCredentialId nao pertence ao dono da API key.
DUPLICATE_LABEL	409	Ja existe credencial com esse label para essa plataforma.
CREDENTIAL_IN_USE	409	Credencial referenciada por job existente; nao pode ser deletada.
RUN_NOT_READY	409	SSE: workflow ainda nao iniciou. Tente novamente em instantes.
RUN_NOT_FOUND	404	SSE: runId desconhecido pelo workflow runtime.
UPLOAD_FAILED	400	Erro generico durante o salvamento de arquivos.

7. Receita completa

Roteiro fim-a-fim para integrar o Hermes em uma rotina noturna de envio de lotes:

Passo 1 — Variáveis

```
BASE=http://localhost:3000
```

```
KEY=hapi_xxxxxxxx... # gerada uma vez na UI
```

Passo 2 — Cadastrar a credencial Orizon (uma vez)

```
CRED_ID=$(
  curl -s -X POST $BASE/api/v1/platform-credentials \
    -H "Authorization: Bearer $KEY" \
    -H "Content-Type: application/json" \
    -d '{"platformId":"orizon_fature","label":"Clinica X",
        "username":"186870","password":"SEGREDO"}' \
    | jq -r '.credential.id'
)
```

Passo 3 — Disparar um job auto-aprovado

```
JOB=$(
  curl -s -X POST $BASE/api/v1/jobs \
    -H "Authorization: Bearer $KEY" \
    -F "flowType=short" \
    -F "platformCredentialId=$CRED_ID" \
    -F "file=@./lote_1124.zip" \
    -F "file=@./lote_1122.zip" \
    | jq -r '.jobId'
)
echo "Job criado: $JOB"
```

Passo 4 — Polling até o job terminar

```
while true; do
  STATUS=$(
    curl -s -H "Authorization: Bearer $KEY" \
      $BASE/api/v1/jobs/$JOB | jq -r '.job.status'
  )
  echo "$STATUS"
  case "$STATUS" in
    login_succeeded|failed) break ;;
  esac
  sleep 5
done
```

Alternativa — SSE em tempo real

```
curl -N -H "Authorization: Bearer $KEY" \
  $BASE/api/v1/jobs/$JOB/events
```

8. Referência OpenAPI

O contrato formal vive no arquivo `docs/openapi.yaml` (OpenAPI 3.1) na raiz do repositório. Use-o para:

- gerar SDKs com `openapi-generator-cli`;
- importar no Postman / Insomnia / Bruno;
- validar requests/responses em testes contract-first.

O spec define todos os schemas (`Job`, `JobEvent`, `TissDocument`, `PlatformCredential`, etc.), os enums (`JobStatus`, `JobFlowType`, `PlatformId`) e as respostas de erro padronizadas. Em caso de divergência entre este guia e o YAML, o YAML é a fonte da verdade.